

LECTURE 4

CEN 348-Artificial Intelligence Systems

Mining Frequent Itemsets – Apriori Algorithm

- In data mining, association rule learning is a popular and well researched method for discovering interesting relations between variables in large databases.
- Piatetsky-Shapiro describes analyzing and presenting strong rules discovered in databases using different measures of interestingness.
- Based on the concept of strong rules, Agrawal introduced association rules for discovering regularities between products in large scale transaction data recorded by point-of-sale (POS) systems in supermarkets.
- For example, the rule $\{\text{onion}, \text{potatoes}\} \Rightarrow \{\text{burger}\}$ found in the sales data of a supermarket would indicate that if a customer buys onions and potatoes together, he or she is likely to also buy burger. Such information can be used as the basis for decisions about marketing activities such as, e.g., promotional pricing or product placements.

Mining Frequent Itemsets – Apriori Algorithm

- In addition to the above example from market basket analysis association rules are employed today in many application areas including Web usage mining, intrusion detection and bioinformatics.
- In computer science and data mining, Apriori is a classic algorithm for learning association rules.
- Apriori is designed to operate on databases containing transactions (for example, collections of items bought by customers, or details of a website frequentation). Other algorithms are designed for finding association rules in data having no transactions (Winepi and Minepi), or having no timestamps (DNA sequencing).

Mining Frequent Itemsets – Apriori Algorithm

Definition:

Following the original definition by Agrawal the problem of association rule mining is defined as:

Let $\mathbf{I} = \{i_1, i_2, \dots, i_n\}$ be a set of n binary attributes called *items*. Let $\mathbf{D} = \{t_1, t_2, \dots, t_n\}$ be a set of transactions called the *database*. Each transaction in $\mathbf{D}$ has a unique transaction ID and contains a subset of the items in $\mathbf{I}$. A *rule* is defined as an implication of the form $\mathbf{X} \rightarrow \mathbf{Y}$ where $\mathbf{X}, \mathbf{Y} \subseteq \mathbf{I}$ and $\mathbf{X} \cap \mathbf{Y} = \emptyset$. The sets of items (for short *itemsets*) $\mathbf{X}$ and $\mathbf{Y}$ are called *antecedent* (left-hand-side or LHS) and *consequent* (right-hand-side or RHS) of the rule respectively.

To illustrate the concepts, we use a small example from the supermarket domain. The set of items is $\mathbf{I} = \{\text{milk}, \text{bread}, \text{butter}, \text{beer}\}$ and a small database containing the items (1 codes presence and 0 absence of an item in a transaction) is shown in the table below. An example rule for the supermarket could be $\{\text{milk}, \text{bread}\} \Rightarrow \{\text{butter}\}$ meaning that if milk and bread is bought, customers also buy butter.

Note: this example is extremely small. In practical applications, a rule needs a support of several hundred transactions before it can be considered statistically significant, and datasets often contain thousands or millions of transactions.

Transaction ID	milk	Bread	butter	beer
1	1	1	0	0
2	0	1	1	0
3	0	0	0	1
4	1	1	1	0
5	0	1	0	0
6	1	0	0	0
7	0	1	1	1
8	1	1	1	1
9	0	1	0	1
10	1	1	0	0
11	1	0	0	0
12	0	0	0	1
13	1	1	1	0
14	1	0	1	0
15	1	1	1	1

Mining Frequent Itemsets – Apriori Algorithm

Useful Concepts

To select interesting rules from the set of all possible rules, constraints on various measures of significance and interest can be used. The best-known constraints are minimum thresholds on support and confidence.

Support

The support **supp**(X) of an itemset X is defined as the proportion of transactions in the data set which contain the itemset.

$$\text{supp}(X) = \text{no. of transactions which contain the itemset } X / \text{total no. of transactions}$$

In the example database, the itemset {milk,bread,butter} has a support of $4 / 15 = 0.26$ since it occurs in 26% of all transactions. To be even more explicit we can point out that 4 is the number of transactions from the database which contain the itemset {milk,bread,butter} while 15 represents the total number of transactions.

Mining Frequent Itemsets – Apriori Algorithm

Confidence

The *confidence* of a rule is defined:

$$\mathit{conf}(X \rightarrow Y) = \mathit{supp}(X \cup Y) / \mathit{supp}(X)$$

For the rule {milk,bread} $\Rightarrow$ {butter} we have the following confidence:

$$\mathit{supp}(\{\text{milk,bread,butter}\}) / \mathit{supp}(\{\text{milk,bread}\}) = 0.26 / 0.4 = 0.65$$

This means that for 65% of the transactions containing milk and bread the rule is correct. Confidence can be interpreted as an estimate of the probability $P(Y | X)$, the probability of finding the RHS of the rule in transactions under the condition that these transactions also contain the LHS.

Mining Frequent Itemsets – Apriori Algorithm

Lift

The *lift* of a rule is defined as:

$$lift(X \rightarrow Y) = \frac{supp(X \cup Y)}{supp(Y) * supp(X)}$$

The rule {milk,bread} $\Rightarrow$ {butter} has the following lift:

$$supp(\{milk,bread,butter\}) / supp(\{butter\}) \times supp(\{milk,bread\}) = 0.26/0.46 \times 0.4 = 1.4$$

Mining Frequent Itemsets – Apriori Algorithm

Conviction

The *conviction* of a rule is defined as:

$$\text{conv}(X \rightarrow Y) = \frac{1 - \text{supp}(Y)}{1 - \text{conf}(X \rightarrow Y)}$$

The rule {milk,bread} $\Rightarrow$ {butter} has the following conviction:

$$1 - \text{supp}(\{\text{butter}\}) / 1 - \text{conf}(\{\text{milk,bread}\} \Rightarrow \{\text{butter}\}) = 1 - 0.46 / 1 - 0.65 = 1.54$$

The conviction of the rule $X \Rightarrow Y$ can be interpreted as the ratio of the expected frequency that X occurs without Y (that is to say, the frequency that the rule makes an incorrect prediction) if X and Y were independent divided by the observed frequency of incorrect predictions.

In this example, the conviction value of 1.54 shows that the rule {milk,bread} $\Rightarrow$ {butter} would be incorrect 54% more often (1.54 times as often) if the association between X and Y was purely random chance.

Apriori algorithm

General Process

Association rule generation is usually split up into two separate steps:

1. First, minimum support is applied to find all *frequent itemsets* in a database.
2. Second, these frequent itemsets and the minimum confidence constraint are used to form rules.

While the second step is straight forward, the first step needs more attention.

Finding all frequent itemsets in a database is difficult since it involves searching all possible itemsets (item combinations). The set of possible itemsets is the power set over I and has size $2^n - 1$ (excluding the empty set which is not a valid itemset). Although the size of the powerset grows exponentially in the number of items n in I , efficient search is possible using the ***downward-closure property*** of support (also called *anti-monotonicity*) which guarantees that for a frequent itemset, all its subsets are also frequent and thus for an infrequent itemset, all its supersets must also be infrequent. Exploiting this property, efficient algorithms (e.g., Apriori and Eclat) can find all frequent itemsets.

Apriori Algorithm Pseudocode

```
procedure Apriori ( $T$ ,  $minSupport$ ) { //  $T$  is the database and  $minSupport$  is the minimum support
     $L_1 = \{ \text{frequent items} \};$ 
    for ( $k = 2; L_{k-1} \neq \emptyset; k++$ ) {
         $C_k =$  candidates generated from  $L_{k-1}$ 
        // that is cartesian product  $L_{k-1} \times L_{k-1}$  and eliminating any  $k-1$  size itemset that is not
        // frequent
        for each transaction  $t$  in database do{
            #increment the count of all candidates in  $C_k$  that are contained in  $t$ 
             $L_k =$  candidates in  $C_k$  with  $minSupport$ 
        } // end for each
    } // end for
    return  $\bigcup_k L_k$ ;
}
```

As is common in association rule mining, given a set of *itemsets* (for instance, sets of retail transactions, each listing individual items purchased), the algorithm attempts to find subsets which are common to at least a minimum number C of the itemsets. Apriori uses a "bottom up" approach, where frequent subsets are extended one item at a time (a step known as *candidate generation*), and groups of candidates are tested against the data. The algorithm terminates when no further successful extensions are found.

The Apriori Algorithm: Basics

Key Concepts :

- **Frequent Itemsets**: The sets of item which has minimum support (denoted by L_i for i^{th} -Itemset).
- **Apriori Property**: Any subset of frequent itemset must be frequent.
- **Join Operation**: To find L_k , a set of candidate k-itemsets is generated by joining L_{k-1} with itself.

The Apriori Algorithm in a Nutshell

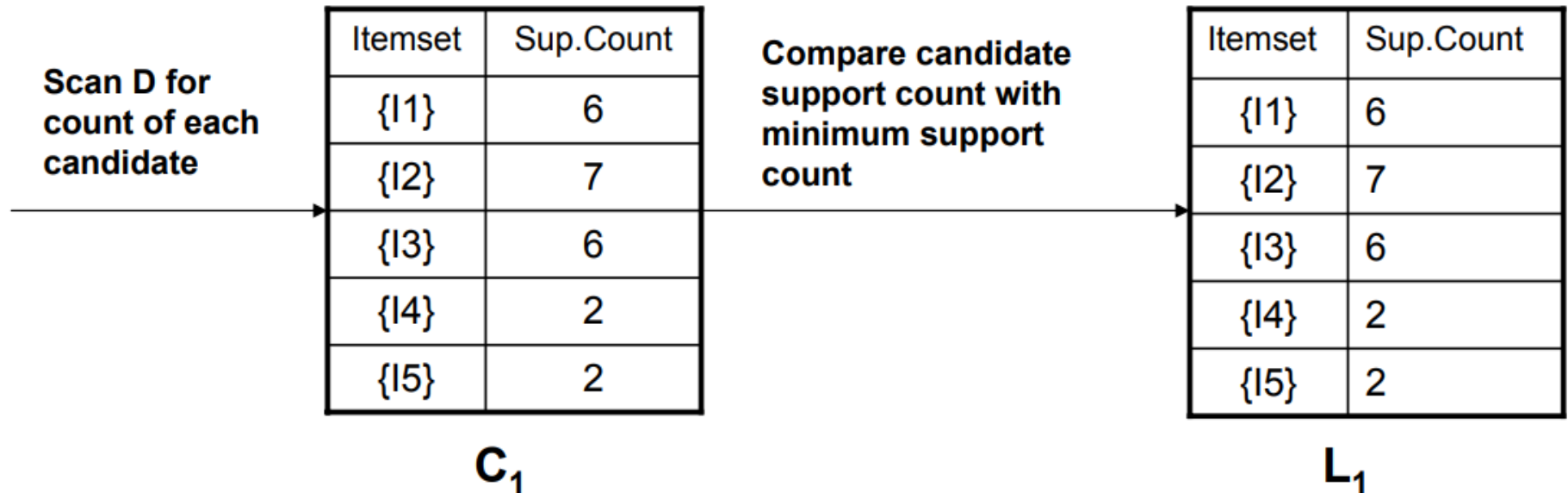
- Find the *frequent itemsets*: the sets of items that have minimum support
 - A subset of a frequent itemset must also be a frequent itemset
 - i.e., if $\{AB\}$ is a frequent itemset, both $\{A\}$ and $\{B\}$ should be a frequent itemset
 - Iteratively find frequent itemsets with cardinality from 1 to k (k -itemset)
- Use the frequent itemsets to generate association rules.

The Apriori Algorithm: Example

TID	List of Items
T100	I1, I2, I5
T100	I2, I4
T100	I2, I3
T100	I1, I2, I4
T100	I1, I3
T100	I2, I3
T100	I1, I3
T100	I1, I2, I3, I5
T100	I1, I2, I3

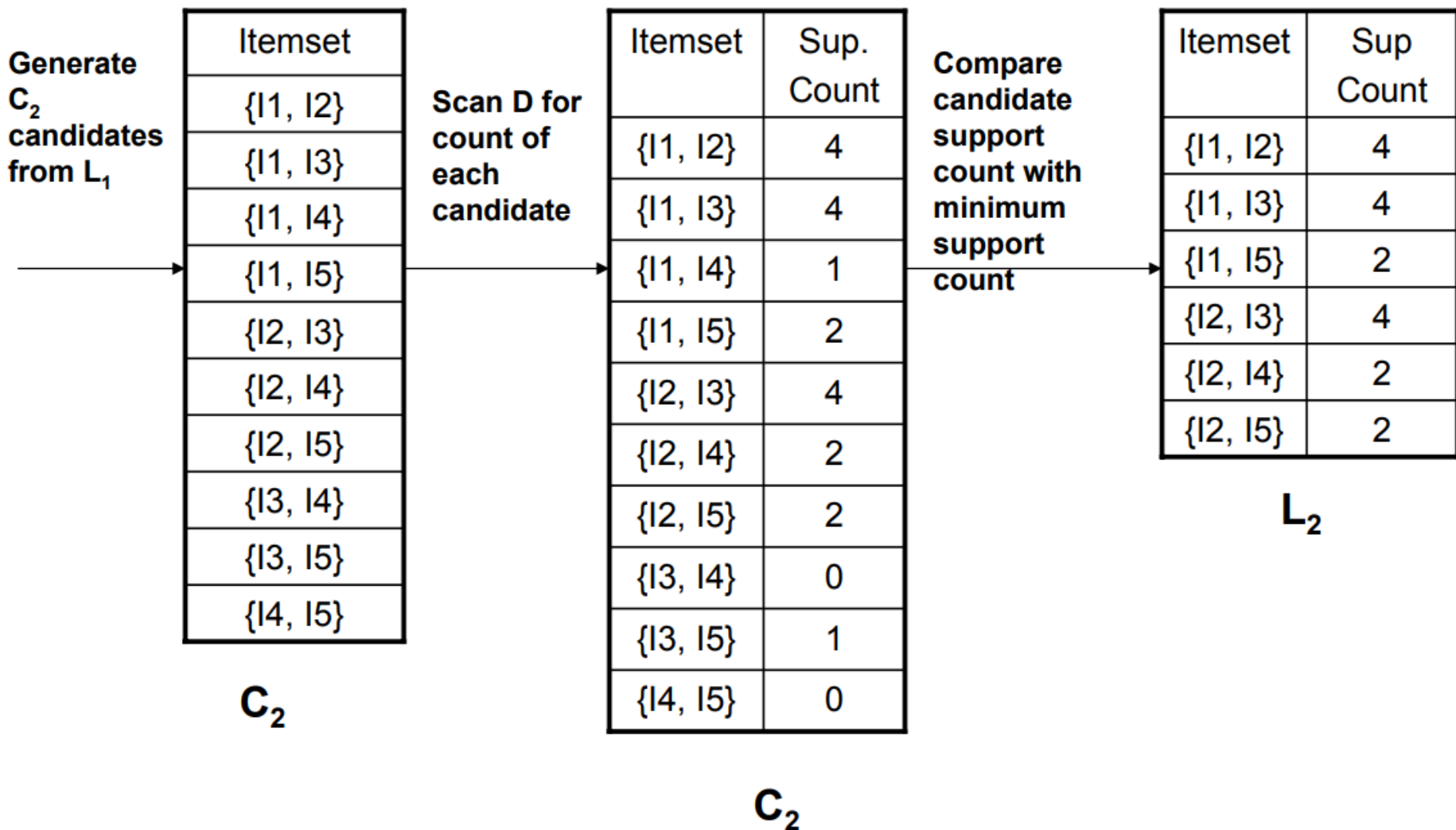
- Consider a database, D , consisting of 9 transactions.
- Suppose min. support count required is 2 (i.e. $\text{min_sup} = 2/9 = 22\%$)
- Let minimum confidence required is 70%.
- We have to first find out the frequent itemset using Apriori algorithm.
- Then, Association rules will be generated using min. support & min. confidence.

Step 1: Generating 1-itemset Frequent Pattern



- The set of frequent 1-itemsets, L_1 , consists of the candidate 1-itemsets satisfying minimum support.
- In the first iteration of the algorithm, each item is a member of the set of candidate.

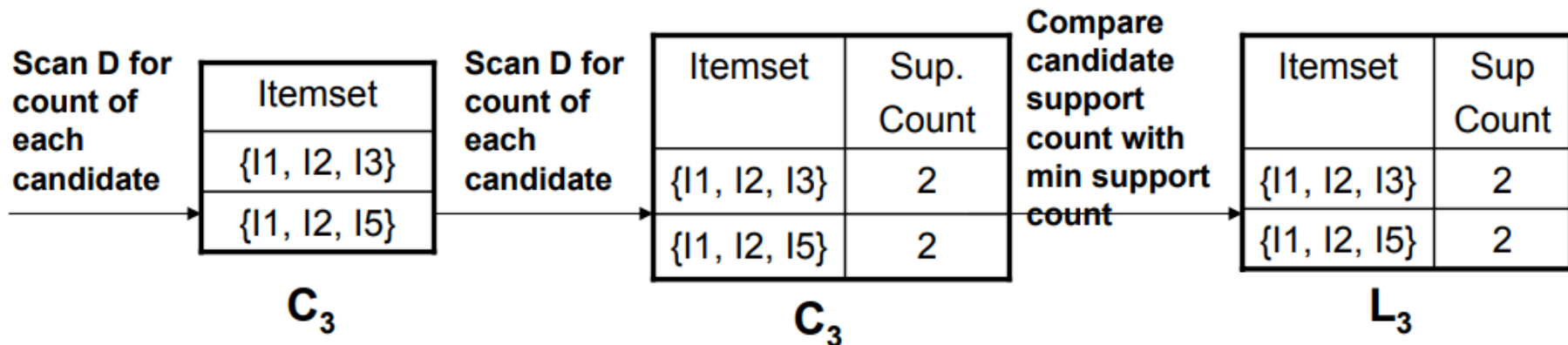
Step 2: Generating 2-itemset Frequent Pattern



Step 2: Generating 2-itemset Frequent Pattern

- To discover the set of frequent 2-itemsets, L_2 , the algorithm uses $L_1 \text{ Join } L_1$ to generate a candidate set of 2-itemsets, C_2 .
- Next, the transactions in D are scanned and the support count for each candidate itemset in C_2 is accumulated (as shown in the middle table).
- The set of frequent 2-itemsets, L_2 , is then determined, consisting of those candidate 2-itemsets in C_2 having minimum support.
- **Note:** We haven't used Apriori Property yet.

Step 3: Generating 3-itemset Frequent Pattern



- The generation of the set of candidate 3-itemsets, C_3 , involves use of the Apriori Property.
- In order to find C_3 , we compute $L_2 \text{ Join } L_2$.
- $C_3 = L_2 \text{ Join } L_2 = \{\{l1, l2, l3\}, \{l1, l2, l5\}, \{l1, l3, l5\}, \{l2, l3, l4\}, \{l2, l3, l5\}, \{l2, l4, l5\}\}$.
- Now, Join step is complete and Prune step will be used to reduce the size of C_3 . Prune step helps to avoid heavy computation due to large C_k .

Step 3: Generating 3-itemset Frequent Pattern

- Based on the **Apriori property** that all subsets of a frequent itemset must also be frequent, we can determine that four latter candidates cannot possibly be frequent. How ?
- For example , lets take $\{I1, I2, I3\}$. The 2-item subsets of it are $\{I1, I2\}$, $\{I1, I3\}$ & $\{I2, I3\}$. Since all 2-item subsets of $\{I1, I2, I3\}$ are members of L_2 , We will keep $\{I1, I2, I3\}$ in C_3 .
- Lets take another example of $\{I2, I3, I5\}$ which shows how the pruning is performed. The 2-item subsets are $\{I2, I3\}$, $\{I2, I5\}$ & $\{I3, I5\}$.
- BUT, $\{I3, I5\}$ is not a member of L_2 and hence it is not frequent **violating Apriori Property**. Thus We will have to remove $\{I2, I3, I5\}$ from C_3 .
- Therefore, $C_3 = \{\{I1, I2, I3\}, \{I1, I2, I5\}\}$ after checking for all members of **result of Join operation** for **Pruning**.
- Now, the transactions in D are scanned in order to determine L_3 , **consisting of those candidates 3-itemsets in C_3 having minimum support**.

Step 4: Generating 4-itemset Frequent Pattern

- The algorithm uses L_3 Join L_3 to generate a candidate set of 4-itemsets, C_4 . Although the join results in $\{\{I1, I2, I3, I5\}\}$, this itemset is pruned since its subset $\{\{I2, I3, I5\}\}$ is not frequent.
- Thus, $C_4 = \varnothing$, and algorithm terminates, having found all of the frequent items. This completes our Apriori Algorithm.
- What's Next ?
These frequent itemsets will be used to generate strong association rules (where strong association rules satisfy both minimum support & minimum confidence).

Step 5: Generating Association Rules from Frequent Itemsets

- Procedure:

- For each frequent itemset I , generate all nonempty subsets of I .
- For every nonempty subset s of I , output the rule " $s \rightarrow (I-s)$ " if $\text{support_count}(I) / \text{support_count}(s) \geq \text{min_conf}$ where min_conf is minimum confidence threshold.

- Back To Example:

We had $L = \{\{I1\}, \{I2\}, \{I3\}, \{I4\}, \{I5\}, \{I1, I2\}, \{I1, I3\}, \{I1, I5\}, \{I2, I3\}, \{I2, I4\}, \{I2, I5\}, \{I1, I2, I3\}, \{I1, I2, I5\}\}$.

- Let's take $I = \{I1, I2, I5\}$.
- Its all nonempty subsets are $\{I1, I2\}, \{I1, I5\}, \{I2, I5\}, \{I1\}, \{I2\}, \{I5\}$.

Step 5: Generating Association Rules from Frequent Itemsets

- Let minimum confidence threshold is , say 70%.
- The resulting association rules are shown below, each listed with its confidence.
 - R1: $I1 \wedge I2 \rightarrow I5$
 - Confidence = $sc\{I1, I2, I5\} / sc\{I1, I2\} = 2/4 = 50\%$
 - R1 is Rejected.
 - R2: $I1 \wedge I5 \rightarrow I2$
 - Confidence = $sc\{I1, I2, I5\} / sc\{I1, I5\} = 2/2 = 100\%$
 - R2 is Selected.
 - R3: $I2 \wedge I5 \rightarrow I1$
 - Confidence = $sc\{I1, I2, I5\} / sc\{I2, I5\} = 2/2 = 100\%$
 - R3 is Selected.

Step 5: Generating Association Rules from Frequent Itemsets

- R4: $I1 \rightarrow I2 \wedge I5$
 - Confidence = $sc\{I1, I2, I5\} / sc\{I1\} = 2/6 = 33\%$
 - R4 is Rejected.
- R5: $I2 \rightarrow I1 \wedge I5$
 - Confidence = $sc\{I1, I2, I5\} / \{I2\} = 2/7 = 29\%$
 - R5 is Rejected.
- R6: $I5 \rightarrow I1 \wedge I2$
 - Confidence = $sc\{I1, I2, I5\} / \{I5\} = 2/2 = 100\%$
 - R6 is Selected.

In this way, We have found three strong association rules.

Linear Regression with Multiple Variables

Linear regression with multiple variables is also known as “multivariate linear regression”.

We now introduce notation for equations where we can have any number of input variables.

$x_j^{(i)}$ = value of feature j in the i^{th} training example

$x^{(i)}$ = the column vector of all the feature inputs of the i^{th} training example

m = the number of training examples

$n = |x^{(i)}|$; (the number of features)

Now define the multivariate form of the hypothesis function as follows, accommodating these multiple features:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \dots + \theta_n x_n$$

Linear Regression with Multiple Variables

In order to develop intuition about this function, we can think about θ_0 as the basic price of a house, θ_1 as the price per square meter, θ_2 as the price per floor, etc. x_1 will be the number of square meters in the house, x_2 the number of floors, etc.

Using the definition of matrix multiplication, our multivariate hypothesis function can be concisely represented as:

$$h_{\theta}(x) = [\theta_0 \quad \theta_1 \dots \theta_n] \begin{bmatrix} x_0 \\ x_1 \\ \dots \\ x_n \end{bmatrix} = \theta^T x$$

$$x_0^{(i)} = 1 \text{ for } (i \in 1, \dots, m)$$

[Note: So that we can do matrix operations with theta and x, we will set $x_{(0)}^{(i)} = 1$, for all values of i. This makes the two vectors 'theta' and $x_{(i)}$ match each other element-wise (that is, have the same number of elements: n+1).]

The training examples are stored in X row-wise, like such:

$$\begin{bmatrix} x_{(0)}^{(1)} & x_{(1)}^{(1)} \\ x_{(0)}^{(2)} & x_{(1)}^{(2)} \\ x_{(0)}^{(3)} & x_{(1)}^{(3)} \end{bmatrix}, \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}$$

You can calculate the hypothesis as a column vector of size (m x 1) with:

$$h_{\theta}(X) = X\theta$$

Linear Regression with Multiple Variables

Cost Function

For the parameter vector θ (of type $\mathbb{R}^{n+1}$ or in $\mathbb{R}^{(n+1) \times 1}$, the cost function is:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)^2$$

The vectorized version is:

$$J(\theta) = \frac{1}{2m} (X\theta - \bar{y})^T (X\theta - \bar{y})$$

where $\bar{y}$ denotes the vector of all y values

Linear Regression with Multiple Variables

Gradient Descent for Multiple Variables

The gradient descent equation itself is generally the same form; we just have to repeat it for our 'n' features:

repeat until convergence: {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) * x_0^{(i)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) * x_1^{(i)}$$

$$\theta_2 := \theta_2 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) * x_2^{(i)}$$

...

}

Linear Regression with Multiple Variables

In other words:

repeat until convergence: {

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) * x_0^{(i)} \text{ for } j := 0..n$$

...

}

Matrix Notation

The Gradient Descent rule can be expressed as:

$$\theta := \theta - \alpha \nabla J(\theta)$$

where $\nabla J(\theta)$ is a column vector of the form:

$$\nabla J(\theta) = \begin{bmatrix} \frac{\partial J(\theta)}{\partial \theta_0} \\ \frac{\partial J(\theta)}{\partial \theta_1} \\ \cdot \\ \cdot \\ \cdot \\ \frac{\partial J(\theta)}{\partial \theta_n} \end{bmatrix}$$

Linear Regression with Multiple Variables

The j-th component of the gradient is the summation of the product of two terms:

$$\begin{aligned}\frac{\partial J(\theta)}{\partial \theta_j} &= \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) * x_j^{(i)} \\ &= \frac{1}{m} \sum_{i=1}^m x_j^{(i)} * (h_{\theta}(x^{(i)}) - y^{(i)})\end{aligned}$$

Sometimes, the summation of the product of two terms can be expressed as the product of two vectors.

Here, $x_j^{(i)}$, for $i = 1, \dots, m$, represents the m elements of the j -th column, $\bar{x}_j$, of the training set X .

The other term $(h_{\theta}(x^{(i)}) - y^{(i)})$ is the vector of the deviations between the predictions $h_{\theta}(x^{(i)})$ and the true values $y^{(i)}$. Re-writing $\frac{\partial J(\theta)}{\partial \theta_j}$ we have:

$$\begin{aligned}\frac{\partial J(\theta)}{\partial \theta_j} &= \frac{1}{m} \bar{x}_j^T (X\theta - \bar{y}) \\ \nabla J(\theta) &= \frac{1}{m} X^T (X\theta - \bar{y})\end{aligned}$$

Finally, the matrix notation (vectorized) of the Gradient Descent rule is:

$$\theta := \theta - \frac{\alpha}{m} X^T (X\theta - \bar{y})$$

Feature Normalization

We can speed up gradient descent by having each of our input values in roughly the same range. This is because θ will descend quickly on small ranges and slowly on large ranges, and so will oscillate inefficiently down to the optimum when the variables are very uneven.

The way to prevent this is to modify the ranges of our input variables so that they are all roughly the same. Ideally:

$$-1 \leq x_{(i)} \leq 1$$

or

$$-0.5 \leq x_{(i)} \leq 0.5$$

These aren't exact requirements; we are only trying to speed things up. The goal is to get all input variables into roughly one of these ranges, give or take a few.

Two techniques to help with this are feature scaling and mean normalization. Feature scaling involves dividing the input values by the range (i.e. the maximum value minus the minimum value) of the input variable, resulting in a new range of just 1. Mean normalization involves subtracting the average value for an input variable from the values for that input variable, resulting in a new average value for the input variable of just zero. To implement both of these techniques, adjust your input values as shown in this formula:

$$x_i := \frac{x_i - \mu_i}{s_i}$$

Where μ_i is the average of all the values for feature (i) and s_i is the range of values (max - min), or s_i is the standard deviation.

Note that dividing by the range, or dividing by the standard deviation, give different results. The quizzes in this course use range - the programming exercises use standard deviation.

Example: x_i is housing prices with range of 100 to 2000, with a mean value of 1000. Then,

$$x_i := \frac{\text{price} - 1000}{1900}$$

Difference between Batch Gradient Descent and Stochastic Gradient Descent

S.NO.	Batch Gradient Descent	Stochastic Gradient Descent
1.	Computes gradient using the whole Training sample	Computes gradient using a single Training sample
2.	Slow and computationally expensive algorithm	Faster and less computationally expensive than Batch GD
3.	Not suggested for huge training samples.	Can be used for large training samples.
4.	Deterministic in nature.	Stochastic in nature.
5.	Gives optimal solution given sufficient time to converge.	Gives good solution but not optimal.
6.	No random shuffling of points are required.	The data sample should be in a random order, and this is why we want to shuffle the training set for every epoch.
7.	Can't escape shallow local minima easily.	SGD can escape shallow local minima more easily.
8.	Convergence is slow.	Reaches the convergence much faster.

Logistic Regression

- we introduce an algorithm that is admirably suited for discovering the link between features and some particular outcome: logistic regression.
- Indeed, logistic regression is one of the most important analytic tools in the social and natural sciences.
- Logistic regression is the baseline supervised machine learning algorithm for classification, and also has a very close relationship with neural networks.
- A neural network can be viewed as a series of logistic regression classifiers stacked on top of each other.
- Thus the classification and machine learning techniques introduced here will play an important role throughout the book.

Logistic Regression

- Logistic regression can be used to classify an observation into one of two classes (like ‘positive sentiment’ and ‘negative sentiment’), or into one of many classes.
- Because the mathematics for the two-class case is simpler, we’ll describe this special case of logistic regression.

Components of a probabilistic machine learning classifier: Like naive Bayes, logistic regression is a probabilistic classifier that makes use of supervised machine learning. Machine learning classifiers require a training corpus of m input/output pairs $(x^{(i)}, y^{(i)})$. (We'll use superscripts in parentheses to refer to individual instances in the training set—for sentiment classification each instance might be an individual document to be classified). A machine learning system for classification then has four components:

1. A **feature representation** of the input. For each input observation $x^{(i)}$, this will be a vector of features $[x_1, x_2, \dots, x_n]$. We will generally refer to feature i for input $x^{(j)}$ as $x_i^{(j)}$, sometimes simplified as x_i , but we will also see the notation f_i , $f_i(x)$, or, for multiclass classification, $f_i(c, x)$.
2. A classification function that computes $\hat{y}$, the estimated class, via $p(y|x)$. In the next section we will introduce the **sigmoid** and **softmax** tools for classification.
3. An objective function for learning, usually involving minimizing error on training examples. We will introduce the **cross-entropy loss function**.
4. An algorithm for optimizing the objective function. We introduce the **stochastic gradient descent** algorithm.

Logistic regression has two phases:

training: we train the system (specifically the weights w and b) using stochastic gradient descent and the cross-entropy loss.

test: Given a test example x we compute $p(y|x)$ and return the higher probability label $y = 1$ or $y = 0$.

Classification: the sigmoid

The goal of binary logistic regression is to train a classifier that can make a binary decision about the class of a new input observation. Here we introduce the **sigmoid** classifier that will help us make this decision.

Consider a single input observation x , which we will represent by a vector of features $[x_1, x_2, \dots, x_n]$ (we'll show sample features in the next subsection). The classifier output y can be 1 (meaning the observation is a member of the class) or 0 (the observation is not a member of the class). We want to know the probability $P(y = 1|x)$ that this observation is a member of the class. So perhaps the decision is “positive sentiment” versus “negative sentiment”, the features represent counts of words in a document, $P(y = 1|x)$ is the probability that the document has positive sentiment, and $P(y = 0|x)$ is the probability that the document has negative sentiment.

Logistic regression solves this task by learning, from a training set, a vector of **weights** and a **bias term**. Each weight w_i is a real number, and is associated with one of the input features x_i . The weight w_i represents how important that input feature is to the classification decision, and can be positive (providing evidence that the instance being classified belongs in the positive class) or negative (providing evidence that the instance being classified belongs in the negative class). Thus we might expect in a sentiment task the word *awesome* to have a high positive weight, and *abysmal* to have a very negative weight. The **bias term**, also called the **intercept**, is another real number that's added to the weighted inputs.

To make a decision on a test instance— after we’ve learned the weights in training— the classifier first multiplies each x_i by its weight w_i , sums up the weighted features, and adds the bias term b . The resulting single number z expresses the weighted sum of the evidence for the class.

$$z = \left(\sum_{i=1}^n w_i x_i \right) + b \quad (5.2)$$

In the rest of the book we’ll represent such sums using the **dot product** notation from linear algebra. The dot product of two vectors a and b , written as $a \cdot b$ is the sum of the products of the corresponding elements of each vector. Thus the following is an equivalent formation to Eq. 5.2:

$$z = w \cdot x + b \quad (5.3)$$

But note that nothing in Eq. 5.3 forces z to be a legal probability, that is, to lie between 0 and 1. In fact, since weights are real-valued, the output might even be negative; z ranges from $-\infty$ to ∞ .

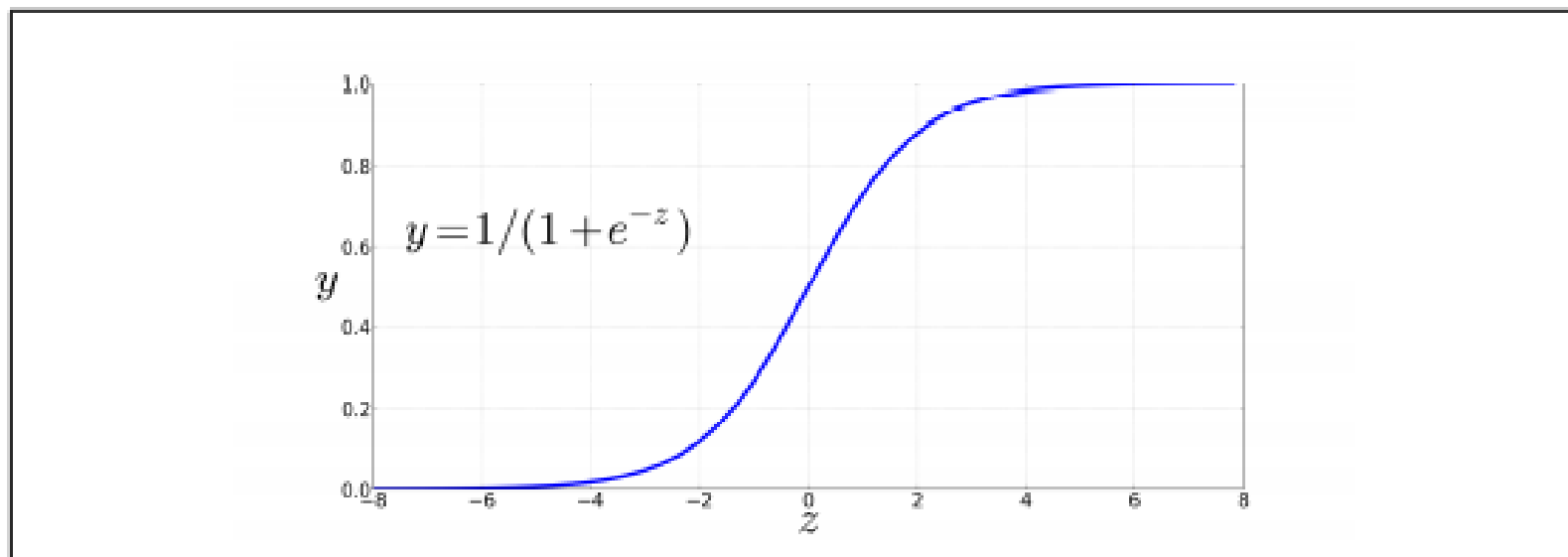


Figure 5.1 The sigmoid function $y = \frac{1}{1+e^{-z}}$ takes a real value and maps it to the range $[0, 1]$. It is nearly linear around 0 but outlier values get squashed toward 0 or 1.

To create a probability, we'll pass z through the **sigmoid** function, $\sigma(z)$. The sigmoid function (named because it looks like an s) is also called the **logistic function**, and gives logistic regression its name. The sigmoid has the following equation, shown graphically in Fig. 5.1:

$$y = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + \exp(-z)} \quad (5.4)$$

(For the rest of the book, we'll use the notation $\exp(x)$ to mean e^x .) The sigmoid has a number of advantages; it takes a real-valued number and maps it into the range $[0, 1]$, which is just what we want for a probability. Because it is nearly linear around 0 but flattens toward the ends, it tends to squash outlier values toward 0 or 1. And it's differentiable, which as we'll see in Section 5.8 will be handy for learning.

We're almost there. If we apply the sigmoid to the sum of the weighted features, we get a number between 0 and 1. To make it a probability, we just need to make sure that the two cases, $p(y = 1)$ and $p(y = 0)$, sum to 1. We can do this as follows:

$$\begin{aligned}P(y = 1) &= \sigma(w \cdot x + b) \\&= \frac{1}{1 + \exp(-(w \cdot x + b))} \\P(y = 0) &= 1 - \sigma(w \cdot x + b) \\&= 1 - \frac{1}{1 + \exp(-(w \cdot x + b))} \\&= \frac{\exp(-(w \cdot x + b))}{1 + \exp(-(w \cdot x + b))}\end{aligned}\tag{5.5}$$

The sigmoid function has the property

$$1 - \sigma(x) = \sigma(-x)\tag{5.6}$$

so we could also have expressed $P(y = 0)$ as $\sigma(-(w \cdot x + b))$.

Now we have an algorithm that given an instance x computes the probability $P(y = 1|x)$. How do we make a decision? For a test instance x , we say yes if the probability $P(y = 1|x)$ is more than .5, and no otherwise. We call .5 the **decision boundary**:

$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

5.1.1 Example: sentiment classification

Let's have an example. Suppose we are doing binary sentiment classification on movie review text, and we would like to know whether to assign the sentiment class $+$ or $-$ to a review document *doc*. We'll represent each input observation by the 6 features $x_1 \dots x_6$ of the input shown in the following table; Fig. 5.2 shows the features in a sample mini test document.

Var	Definition	Value in Fig. 5.2
x_1	count(positive lexicon) $\in$ doc)	3
x_2	count(negative lexicon) $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	log(word count of doc)	$\ln(66) = 4.19$

Let's assume for the moment that we've already learned a real-valued weight for each of these features, and that the 6 weights corresponding to the 6 features are $[2.5, -5.0, -1.2, 0.5, 2.0, 0.7]$, while $b = 0.1$. (We'll discuss in the next section how

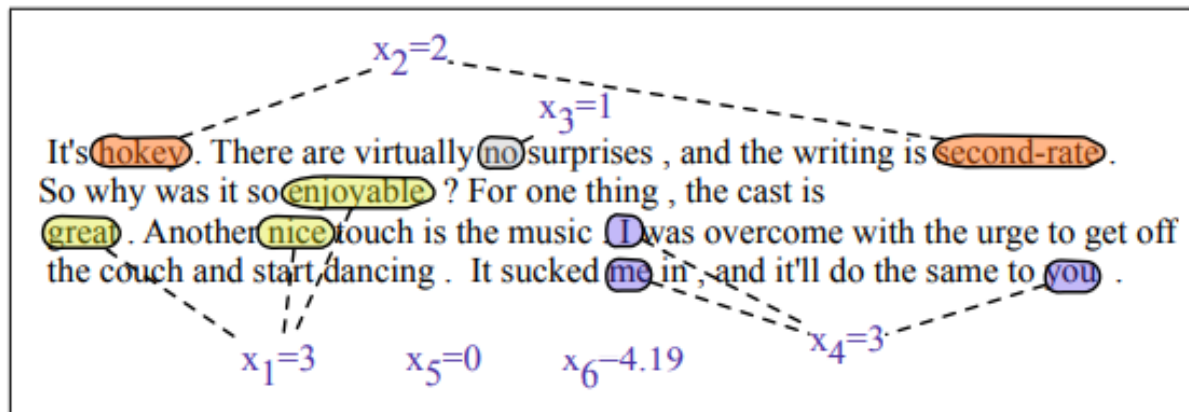


Figure 5.2 A sample mini test document showing the extracted features in the vector x .

the weights are learned.) The weight w_1 , for example indicates how important a feature the number of positive lexicon words (*great*, *nice*, *enjoyable*, etc.) is to a positive sentiment decision, while w_2 tells us the importance of negative lexicon words. Note that $w_1 = 2.5$ is positive, while $w_2 = -5.0$, meaning that negative words are negatively associated with a positive sentiment decision, and are about twice as important as positive words.

Given these 6 features and the input review x , $P(+|x)$ and $P(-|x)$ can be computed using Eq. 5.5:

$$\begin{aligned}
 p(+|x) &= P(Y = 1|x) = \sigma(w \cdot x + b) \\
 &= \sigma([2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \cdot [3, 2, 1, 3, 0, 4.19] + 0.1) \\
 &= \sigma(.833) \\
 &= 0.70 \\
 p(-|x) &= P(Y = 0|x) = 1 - \sigma(w \cdot x + b) \\
 &= 0.30
 \end{aligned} \tag{5.7}$$

Learning in Logistic Regression

How are the parameters of the model, the weights w and bias b , learned? Logistic regression is an instance of supervised classification in which we know the correct label y (either 0 or 1) for each observation x . What the system produces via Eq. 5.5 is $\hat{y}$, the system's estimate of the true y . We want to learn parameters (meaning w and b) that make $\hat{y}$ for each training observation as close as possible to the true y .

This requires two components that we foreshadowed in the introduction to the chapter. The first is a metric for how close the current label ($\hat{y}$) is to the true gold label y . Rather than measure similarity, we usually talk about the opposite of this: the *distance* between the system output and the gold output, and we call this distance the **loss** function or the **cost function**. In the next section we'll introduce the loss function that is commonly used for logistic regression and also for neural networks.

the **cross-entropy loss**.

The second thing we need is an optimization algorithm for iteratively updating the weights so as to minimize this loss function. The standard algorithm for this is **gradient descent**; we'll introduce the **stochastic gradient descent** algorithm in the following section.

The cross-entropy loss function

We need a loss function that expresses, for an observation x , how close the classifier output ($\hat{y} = \sigma(w \cdot x + b)$) is to the correct output (y , which is 0 or 1). We'll call this:

$$L(\hat{y}, y) = \text{How much } \hat{y} \text{ differs from the true } y \quad (5.8)$$

We do this via a loss function that prefers the correct class labels of the training examples to be *more likely*. This is called **conditional maximum likelihood estimation**: we choose the parameters w, b that **maximize the log probability of the true y labels in the training data** given the observations x . The resulting loss function is the negative log likelihood loss, generally called the **cross-entropy loss**.

Let's derive this loss function, applied to a single observation x . We'd like to learn weights that maximize the probability of the correct label $p(y|x)$. Since there are only two discrete outcomes (1 or 0), this is a Bernoulli distribution, and we can express the probability $p(y|x)$ that our classifier produces for one observation as the following (keeping in mind that if $y=1$, Eq. 5.9 simplifies to $\hat{y}$; if $y=0$, Eq. 5.9 simplifies to $1 - \hat{y}$):

$$p(y|x) = \hat{y}^y (1 - \hat{y})^{1-y} \quad (5.9)$$

Now we take the log of both sides. This will turn out to be handy mathematically, and doesn't hurt us; whatever values maximize a probability will also maximize the log of the probability:

$$\begin{aligned} \log p(y|x) &= \log [\hat{y}^y (1 - \hat{y})^{1-y}] \\ &= y \log \hat{y} + (1 - y) \log (1 - \hat{y}) \end{aligned} \quad (5.10)$$

Eq. 5.10 describes a log likelihood that should be maximized. In order to turn this into loss function (something that we need to minimize), we'll just flip the sign on Eq. 5.10. The result is the cross-entropy loss L_{CE} :

$$L_{CE}(\hat{y}, y) = -\log p(y|x) = -[y \log \hat{y} + (1 - y) \log (1 - \hat{y})] \quad (5.11)$$

Finally, we can plug in the definition of $\hat{y} = \sigma(w \cdot x + b)$:

$$L_{CE}(\hat{y}, y) = -[y \log \sigma(w \cdot x + b) + (1 - y) \log (1 - \sigma(w \cdot x + b))] \quad (5.12)$$

Gradient Descent

Our goal with gradient descent is to find the optimal weights: minimize the loss function we've defined for the model. In Eq. 5.13 below, we'll explicitly represent the fact that the loss function L is parameterized by the weights, which we'll refer to in machine learning in general as θ (in the case of logistic regression $\theta = w, b$). So the goal is to find the set of weights which minimizes the loss function, averaged over all examples:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^m L_{\text{CE}}(f(x^{(i)}; \theta), y^{(i)}) \quad (5.13)$$

For logistic regression, this loss function is conveniently **convex**. A convex function has just one minimum; there are no local minima to get stuck in, so gradient descent starting from any point is guaranteed to find the minimum. (By contrast, the loss for multi-layer neural networks is non-convex, and gradient descent may get stuck in local minima for neural network training and never find the global optimum.)

Although the algorithm (and the concept of gradient) are designed for direction *vectors*, let's first consider a visualization of the case where the parameter of our system is just a single scalar w , shown in Fig. 5.3.

Given a random initialization of w at some value w^1 , and assuming the loss function L happened to have the shape in Fig. 5.3, we need the algorithm to tell us whether at the next iteration we should move left (making w^2 smaller than w^1) or right (making w^2 bigger than w^1) to reach the minimum.

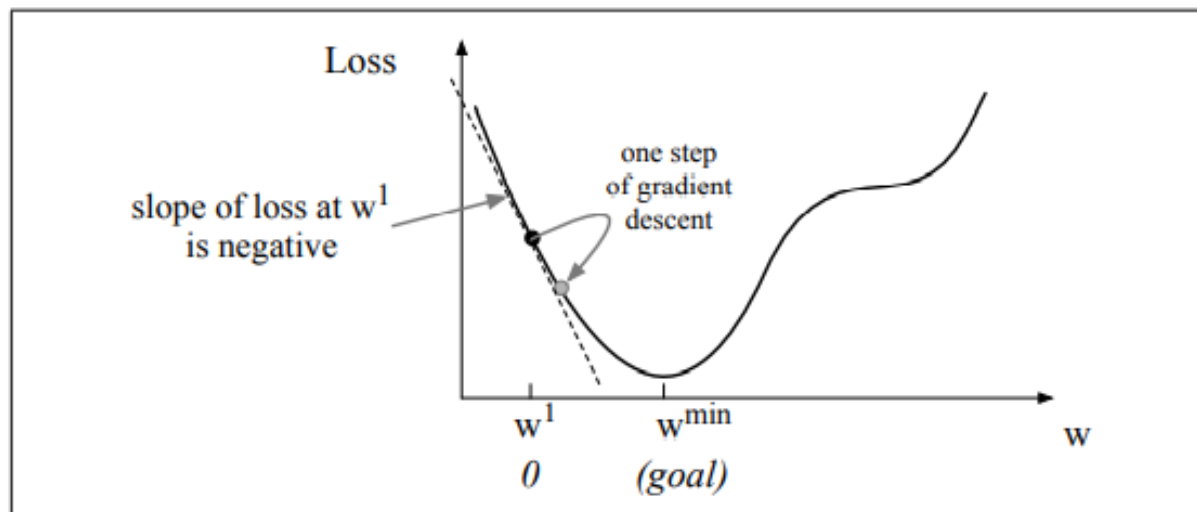


Figure 5.3 The first step in iteratively finding the minimum of this loss function, by moving w in the reverse direction from the slope of the function. Since the slope is negative, we need to move w in a positive direction, to the right. Here superscripts are used for learning steps, so w^1 means the initial value of w (which is 0), w^2 at the second step, and so on.

gradient

The gradient descent algorithm answers this question by finding the **gradient** of the loss function at the current point and moving in the opposite direction. The gradient of a function of many variables is a vector pointing in the direction of the greatest increase in a function. The gradient is a multi-variable generalization of the slope, so for a function of one variable like the one in Fig. 5.3, we can informally think of the gradient as the slope. The dotted line in Fig. 5.3 shows the slope of this hypothetical loss function at point $w = w^1$. You can see that the slope of this dotted line is negative. Thus to find the minimum, gradient descent tells us to go in the opposite direction: moving w in a positive direction.

learning rate

The magnitude of the amount to move in gradient descent is the value of the slope $\frac{d}{dw} f(x; w)$ weighted by a **learning rate** η . A higher (faster) learning rate means that we should move w more on each step. The change we make in our parameter is the

learning rate times the gradient (or the slope, in our single-variable example):

$$w^{t+1} = w^t - \eta \frac{d}{dw} f(x; w) \quad (5.14)$$

In an actual logistic regression, the parameter vector w is much longer than 1 or 2, since the input feature vector x can be quite long, and we need a weight w_i for each x_i . For each dimension/variable w_i in w (plus the bias b), the gradient will have a component that tells us the slope with respect to that variable. Essentially we're asking: "How much would a small change in that variable w_i influence the total loss function L ?"

In each dimension w_i , we express the slope as a partial derivative $\frac{\partial}{\partial w_i}$ of the loss function. The gradient is then defined as a vector of these partials. We'll represent $\hat{y}$ as $f(x; \theta)$ to make the dependence on θ more obvious:

$$\nabla_{\theta} L(f(x; \theta), y) = \begin{bmatrix} \frac{\partial}{\partial w_1} L(f(x; \theta), y) \\ \frac{\partial}{\partial w_2} L(f(x; \theta), y) \\ \vdots \\ \frac{\partial}{\partial w_n} L(f(x; \theta), y) \end{bmatrix} \quad (5.15)$$

The final equation for updating θ based on the gradient is thus

$$\theta_{t+1} = \theta_t - \eta \nabla L(f(x; \theta), y) \quad (5.16)$$

5.4.1 The Gradient for Logistic Regression

In order to update θ , we need a definition for the gradient $\nabla L(f(x; \theta), y)$. Recall that for logistic regression, the cross-entropy loss function is:

$$L_{\text{CE}}(\hat{y}, y) = -[y \log \sigma(w \cdot x + b) + (1 - y) \log (1 - \sigma(w \cdot x + b))] \quad (5.17)$$

It turns out that the derivative of this function for one observation vector x is Eq. 5.18 (the interested reader can see Section 5.8 for the derivation of this equation):

$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y]x_j \quad (5.18)$$

Note in Eq. 5.18 that the gradient with respect to a single weight w_j represents a very intuitive value: the difference between the true y and our estimated $\hat{y} = \sigma(w \cdot x + b)$ for that observation, multiplied by the corresponding input value x_j .

5.4.2 The Stochastic Gradient Descent Algorithm

Stochastic gradient descent is an online algorithm that minimizes the loss function by computing its gradient after each training example, and nudging θ in the right direction (the opposite direction of the gradient). Fig. 5.5 shows the algorithm.

```
function STOCHASTIC GRADIENT DESCENT( $L()$ ,  $f()$ ,  $x$ ,  $y$ ) returns  $\theta$ 
    # where:  $L$  is the loss function
    #  $f$  is a function parameterized by  $\theta$ 
    #  $x$  is the set of training inputs  $x^{(1)}, x^{(2)}, \dots, x^{(m)}$ 
    #  $y$  is the set of training outputs (labels)  $y^{(1)}, y^{(2)}, \dots, y^{(m)}$ 

     $\theta \leftarrow 0$ 
    repeat til done # see caption
        For each training tuple  $(x^{(i)}, y^{(i)})$  (in random order)
            1. Optional (for reporting): # How are we doing on this tuple?
                Compute  $\hat{y}^{(i)} = f(x^{(i)}; \theta)$  # What is our estimated output  $\hat{y}$ ?
                Compute the loss  $L(\hat{y}^{(i)}, y^{(i)})$  # How far off is  $\hat{y}^{(i)}$  from the true output  $y^{(i)}$ ?
            2.  $g \leftarrow \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)})$  # How should we move  $\theta$  to maximize loss?
            3.  $\theta \leftarrow \theta - \eta g$  # Go the other way instead
    return  $\theta$ 
```

Figure 5.5 The stochastic gradient descent algorithm. Step 1 (computing the loss) is used to report how well we are doing on the current tuple. The algorithm can terminate when it converges (or when the gradient norm $< \epsilon$), or when progress halts (for example when the loss starts going up on a held-out set).

The learning rate η is a **hyperparameter** that must be adjusted. If it's too high, the learner will take steps that are too large, overshooting the minimum of the loss function. If it's too low, the learner will take steps that are too small, and take too long to get to the minimum. It is common to start with a higher learning rate and then slowly decrease it, so that it is a function of the iteration k of training; the notation η_k can be used to mean the value of the learning rate at iteration k .

5.4.3 Working through an example

Let's walk through a single step of the gradient descent algorithm. We'll use a simplified version of the example in Fig. 5.2 as it sees a single observation x , whose correct value is $y = 1$ (this is a positive review), and with only two features:

$$\begin{aligned}x_1 &= 3 && \text{(count of positive lexicon words)} \\x_2 &= 2 && \text{(count of negative lexicon words)}\end{aligned}$$

Let's assume the initial weights and bias in θ^0 are all set to 0, and the initial learning rate η is 0.1:

$$\begin{aligned}w_1 = w_2 = b &= 0 \\ \eta &= 0.1\end{aligned}$$

The single update step requires that we compute the gradient, multiplied by the learning rate

$$\theta^{t+1} = \theta^t - \eta \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)})$$

In our mini example there are three parameters, so the gradient vector has 3 dimensions, for w_1 , w_2 , and b . We can compute the first gradient as follows:

$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{CE}(\hat{y},y)}{\partial w_1} \\ \frac{\partial L_{CE}(\hat{y},y)}{\partial w_2} \\ \frac{\partial L_{CE}(\hat{y},y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(w \cdot x + b) - y)x_1 \\ (\sigma(w \cdot x + b) - y)x_2 \\ \sigma(w \cdot x + b) - y \end{bmatrix} = \begin{bmatrix} (\sigma(0) - 1)x_1 \\ (\sigma(0) - 1)x_2 \\ \sigma(0) - 1 \end{bmatrix} = \begin{bmatrix} -0.5x_1 \\ -0.5x_2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$

Now that we have a gradient, we compute the new parameter vector θ^1 by moving θ^0 in the opposite direction from the gradient:

$$\theta^1 = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} - \eta \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix} = \begin{bmatrix} .15 \\ .1 \\ .05 \end{bmatrix}$$

So after one step of gradient descent, the weights have shifted to be: $w_1 = .15$, $w_2 = .1$, and $b = .05$.

Note that this observation x happened to be a positive example. We would expect that after seeing more negative examples with high counts of negative words, that the weight w_2 would shift to have a negative value.